calibration.py

`src/moe_pruning/data/calibration.py · 212 lines · module moe_pruning.data.calibration`

What it does

Reads the calibration dataset (JSONL with `human` and `response` fields), tokenizes it through the model's task objective and splits it deterministically into a training and a validation part.

How it works

- `load_calibration_records()` reads line by line, skips and reports blank lines, invalid JSON, missing or empty fields (with line numbers), honours `CALIBRATION_MAX_SAMPLES` and fails if nothing valid remains.
- `tokenize_calibration()` is the encoder-decoder tokenization: `human` becomes encoder input, `response` the decoder labels; over-long texts are truncated while keeping the EOS token.
- `split_train_validation()` shuffles indices with the configured seed and holds out `BIAS_VALIDATION_FRACTION`.
- `prepare_calibration()` combines everything and records metadata (file SHA-256, sizes, task name) plus a fingerprint. Later stages compare this fingerprint to detect stale artifacts.

Notes

- The same split is used everywhere: EASY-EP importance uses all examples, bias initialisation and training use only the training split, the validation split is held out for monitoring.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class CalibrationDataError`	Raised when the calibration file is missing or has no valid records.
`class CalibrationRecord`	One validated calibration pair (human, response, line number).
`class CalibrationLoadReport`	Counts of loaded and skipped lines plus the first problems found.
`.problem(message)`	Remember a problem message (the first 20 are kept).
`.to_dict()`	Convert to a plain dictionary for JSON reports.
`load_calibration_records(path, max_samples)`	Read and validate a calibration JSONL file line by line.
`class TokenizedPair`	One teacher-forced example: encoder ids and decoder labels (no padding).
`tokenize_calibration(records, tokenizer, max_source_length, max_target_length)`	Tokenize `human` as encoder input and `response` as decoder labels.
`split_train_validation(items, validation_fraction, seed)`	Deterministic shuffle-split; at least one item in each part when possible.
`class PreparedCalibration`	Tokenized calibration data with its deterministic train/validation split.
`.train() [property]`	Examples of the training split.
`.validation() [property]`	Examples of the validation split.
`prepare_calibration(cfg, tokenizer, task)`	Load, tokenize (with the model's task objective) and split the calibration set.

Settings it reads

`BIAS_VALIDATION_FRACTION`, `CALIBRATION_DATASET_PATH`, `CALIBRATION_MAX_SAMPLES`, `MAX_SOURCE_LENGTH`, `MAX_TARGET_LENGTH`, `SEED`

Connections

Uses: `moe_pruning.utils.paths`

Used by: `scripts/collect_stats.py`, `scripts/export_model.py`, `scripts/inspect_model.py`, `scripts/replace_experts.py`, `scripts/train_biases.py`, `src/moe_pruning/tasks/base.py`, `src/moe_pruning/tasks/causal_lm.py`, `src/moe_pruning/tasks/seq2seq_denoising.py`